

What a Query Result Means: Plan-Level Answering over Federated Catalysis Data

Kundai Farai Sachikonye

September 2026

Abstract

The standard route from a chemical question to an answer is: build an ontology, materialise a knowledge graph, apply a reasoner, and issue a SPARQL query. This paper reports what happens when thirteen questions drawn from working biocatalysis practice are put instead to a *plan* language with an explicit capability calculus, and argues that the difference is not one of convenience but of what the returned document is able to say.

The central observation is negative and precise. A SPARQL result set carries one bit of failure information: the solution sequence is either non-empty or empty. Four operationally distinct situations collapse onto that empty result — the corpus genuinely contains no such thing; a source cannot evaluate the constraint at all; an upstream step yielded nothing so a downstream step was never reachable; the request was declined. We show that no rewriting of the query recovers the distinction, because the interface has no channel to carry it. We then exhibit a six-valued verdict assigned by ordered rules over each plan step, a capability check that refuses ill-posed plans *before any request is issued*, and a fourth answer shape — an admissibility statement — for the case where every step succeeds and the plan nonetheless answers a different question than the one asked.

All thirteen questions execute. Five are Doerr’s biocatalysis questions; eight are the generic Chem-DCAT-AP dataset-discovery patterns. Eight return rows. Two are refused statically at `requests_issued = 0`, one of them for data that demonstrably exists in the federation. Two return rows under an explicit admissibility caveat naming the gap. One returns an `empty` that is reported as `empty` and a downstream `starved` that is reported as `starved`, which is the whole point. We give the counterfactual falsification of every constraint — including one that caught a real defect in our own plan, where a checked constraint was never on the path from the question to the payload and the plan returned the right rows for the wrong reason. Sixteen validation checks over the compiler and executor hold. We are explicit about what they do not establish.

Contents

1	The question this paper is about	2
1.1	A single observation	3
1.2	What we propose instead	3
1.3	For the chemist, and for the computer scientist	4
1.4	What this paper does not claim	4
2	Plans, sources, capabilities	5
2.1	The move: a question is a plan, not a query	5
2.2	Sources	5
2.3	Honest declaration, and its asymmetry	6
2.4	Budget and allocation	7

3	Six verdicts, and why two are not enough	7
3.1	The rules	7
3.2	The absent blocker is the point	8
3.3	Starvation is the characteristic federated failure	8
4	Refusal before contact	9
4.1	The whole plan, not the failing step	9
4.2	The refusal names the real obstacle	9
5	When every step succeeds and the answer is still wrong	9
5.1	The problem	9
5.2	Why a reasoner makes this worse	10
5.3	The construct	10
6	Five questions from a working group	11
6.1	Q1: bacterial transaminase without cysteine	11
6.2	Q2: buffer and pH of a specific methyl transfer	11
6.3	Q3: substrate scope and product range	12
6.4	Q4: expected products under stated conditions	12
6.5	Q5: instrument and wavelength for a dated observation	13
6.6	An implementation defect the framework caught in itself	13
7	Eight dataset-discovery patterns	14
7.1	G3 and G8: refused at zero requests	14
7.2	G6: a vocabulary the corpus does not have	14
7.3	G7, and a defect in our own plan	15
7.4	One further contrast worth stating	15
8	Two design decisions that look like omissions	16
8.1	There is no set difference in the plan language	16
8.2	Comparability is a property of a pair, not of a dataset	16
9	Full evidence	17
10	Validation, and what it does not establish	19
11	On not probing a third party’s endpoint	19
12	Related work	20
13	Limitations	21
14	Conclusion	21

1 The question this paper is about

A biocatalysis group keeps its experimental records in a structured form. A colleague asks:

Which biocatalyst, originating from a bacterium and not a eukaryote, catalyses the transamination of benzylethylamine and does not have a cysteine in its protein sequence?

This is a reasonable question and it has, in the group’s data, a determinate answer. The received method for obtaining it is well established and works: express the domain in an ontology, materialise the records as RDF, run a reasoner to close the model under the axioms, and issue a

SPARQL query against the result [Baader et al., 2003, Grau et al., 2008, Harris and Seaborne, 2013]. The Chem-DCAT-AP effort applies exactly this recipe to catalysis data, extending DCAT-AP [European Commission, ISA Programme, 2015, Albertoni et al., 2020] with the chemistry-specific vocabulary the domain needs, and it does so competently.

We are not going to argue that this does not work. It does. We are going to argue that it works in a way that cannot tell you when it has not worked, and that for laboratory data — which is sparse, heterogeneous, and unevenly recorded by construction — that limitation is not incidental.

1.1 A single observation

Consider what the SPARQL engine hands back when the question above returns nothing. It hands back an empty solution sequence. Now consider the situations that produce an empty solution sequence here:

- (a) Every transaminase in the corpus that acts on benzylethylamine has a cysteine. The question has a genuine negative answer.
- (b) The sequence store is a REST service that returns a FASTA record by accession and cannot be asked *which of your records lack residue X*. The constraint was never evaluated.
- (c) No transaminase in the corpus is recorded as bacterial, because organism attribution lives in a source that was not consulted. The cysteine filter had nothing to filter.
- (d) The endpoint declined the request — rate limit, timeout, malformed federation clause.

These are four different facts about the world and about the query apparatus, and they call for four different next actions: publish the negative result; find a different sequence source; add the organism source to the federation; retry. The interface returns the same thing in all four cases.

Observation 1.1 (the one-bit interface). The SPARQL result-set interface has exactly two failure-relevant states: non-empty and empty. Situations (a)–(d) are distinguishable facts about an execution that are mapped onto one state. No rewriting of the query text changes this, because the distinction is not a property of the query — it is a property of the execution, and the interface has no field in which to put it.

This is not a criticism of any implementation. It is a statement about what the abstraction was designed to convey. SPARQL was designed to express *what set of solutions do the data entail*; it answers that question well. It was not designed to convey *how did the attempt to obtain them go*, and asking it to is a category error.

The consequence, however, is borne by the chemist. In a domain where the data are complete — a well-curated single graph, closed-world by construction — case (a) is overwhelmingly the likely reading of an empty result and the ambiguity is harmless. Laboratory catalysis data is the opposite case. Records are partial by nature: a run that did not record its buffer is not a run that used no buffer. Sources are heterogeneous: sequences live in UniProt [The UniProt Consortium, 2023], kinetics in SABIO-RK [Wittig et al., 2018], enzyme classification in BRENDA [Chang et al., 2021], compound identity in ChEBI [Hastings et al., 2016], and the group’s own provenance in its own store. Under those conditions (b), (c), and (d) are not edge cases, they are the common case, and an empty result that cannot be read is the normal outcome.

1.2 What we propose instead

We keep the ontology work. Nothing here argues against having a vocabulary; you cannot ask about a *catalyst* without a term for one. What we change is the layer above: instead of compiling the question into a single query and reading the result set, we compile it into a *plan* — an ordered sequence of steps, each bound to a named source, each carrying a declared set of the capabilities it requires — and we execute the plan against a federation whose sources declare what they can do.

Three things follow, and they are the substance of this paper.

1. **The result carries a verdict per step, drawn from six values, not two** (Section 3). Situations (a)–(d) above receive distinct verdicts, assigned by ordered rules, with a *blocker* naming what stood in the way.
2. **Plans that cannot be answered are refused before contact** (Section 4). If a step requires a capability its source does not declare, the plan is rejected statically and no request is issued — including the requests for the parts that would have succeeded. We demonstrate this at `requests_issued = 0` on two questions, one of which asks for data that provably exists in the federation.
3. **Some questions receive an answer and an admissibility statement** (Section 5). A plan can return the correct rows at every step and still be answering a different question, because the question’s logical form and the corpus’s do not match. Nothing in a verdict can say so, so we say it separately, and we make the plan author name which kind of gap it is.

1.3 For the chemist, and for the computer scientist

This paper has two audiences and they want different things from it. We have tried to serve both rather than splitting the difference.

To the chemist: the claim you should test is in Section 6, where five real questions are worked end to end. The claim is not that we answer more of them — we answer eight of thirteen with rows, which a well-built SPARQL stack over the same data would also do. The claim is that for the five we do not answer, the document we return tells you *why not*, in terms that distinguish a fact about your chemistry from a fact about your infrastructure. Section 6.4 is the one to read first: it is a question about expected products, and the answer is that the corpus contains the reaction and the conditions and still cannot support the inference, for a reason that a reasoner would *hide* rather than expose.

To the computer scientist: the technical content is the capability calculus of Section 2, the verdict rules R1–R6 of Section 3, and the honest-declaration assumption of Section 2.3 — which is the largest gap in the design and which we do not close. The relationship to prior federation work [Wiederhold, 1992, Levy et al., 1996, Haas et al., 1997, Schwarte et al., 2011, Saleem et al., 2016] is that source-capability description is old and well studied; what is new here is treating the *failure* of a capability match as a first-class output rather than as a planner-internal fact, and the observation that this is what the empty-result ambiguity actually needs. The connection to the nulls and certain-answers literature [Codd, 1979, Imielinski and Lipski, 1984, Libkin, 2016] is close and we make it explicit in Section 12.

1.4 What this paper does not claim

We state the limits at the front rather than burying them.

- (i) The capability declarations are *data written by the adapter author*. Nothing verifies them. An over-declaring source produces exactly the silent wrong answer this paper exists to prevent (Section 2.3).
- (ii) The validation suite tests the compiler and the executor against the theory. It does not test the theory against the world (Section 10).
- (iii) The evaluation is over a local fixture built to mirror the structure of the group’s data, not over the live endpoint. This is deliberate and we explain the ethics of it in Section 11.
- (iv) Eight of thirteen questions return rows. We are not claiming higher recall than a conventional stack; on this corpus we would expect the same recall. The claim is about the other five.

2 Plans, sources, capabilities

This section fixes the objects. A reader who wants the chemistry first may skip to Section 6 and return here.

2.1 The move: a question is a plan, not a query

The received pipeline compiles a question into one query and submits it. The query is a single syntactic object whose evaluation is the endpoint’s business, and the planner’s decisions — which source, in what order, with what budget — are either absent (one graph) or internal to a federation engine that does not report them.

We invert this. The user writes a *plan*: an ordered list of named steps, each of which names a source and a predicate, and may bind the output of an earlier step. Queries are *leaves*. The user never writes one. The lowering from a step to a concrete request in the target language is the adapter’s job, and the lowered form is retained in the output so that what was actually sent can be read.

Listing 1: A plan. Every step names its source; no SPARQL appears anywhere.

```
plan mark_q1 {
  budget 12 requests

  let enzymes = from RXN
    ask catalysts_of("RXN:TA-benzylethylamine")
    within 20

  let bacterial = from TAX
    ask typed_as("TAX:bacteria")
    with ?e in enzymes
    within 20

  let no_cys = from SEQ
    ask lacks_residue("C")
    with ?e in candidates
    within 20

  emit no_cys with provenance
}
```

The reason for the inversion is not ergonomics. It is that the information we want in the output — which source could not do what — exists only at the granularity of a step. A single query has one outcome; a plan has one per step, and it is at that granularity that Theorem 1.1 is escapable.

2.2 Sources

Definition 2.1 (source). A source is a quadruple $S = (\eta, \text{cap}, \text{ext}, c)$ where η is a namespace identifier, $\text{cap} \subseteq \mathcal{F}$ is the set of capabilities the source declares, ext is the extraction function mapping a lowered request and its bound inputs to a result set, and $c \in \mathbb{N}$ is the request cost charged against the budget.

Definition 2.2 (capability vocabulary). $\mathcal{F}$ is the fixed, finite set

$$\mathcal{F} = \{\text{pattern, path, filter, bind, agg, neg, order, regex, lookup, link, batch}\}$$

of eleven symbols. Each names an operation a source either can or cannot perform: matching a triple pattern; traversing a path, including backwards; applying a value restriction; accepting a set of bound values as input; aggregating; evaluating negation; ordering; matching a regular expression; retrieving a record by key; following a link; and accepting a batch of keys in one request.

The vocabulary is deliberately small and deliberately closed. Its purpose is not to describe a source completely — it cannot — but to be coarse enough that an adapter author can answer each of eleven yes/no questions honestly. A finer vocabulary would be more expressive and less reliably answered, and the failure mode of a wrong answer here is severe (Section 2.3).

Definition 2.3 (requirement). Each predicate p carries a requirement $\text{req}(p) \subseteq \mathcal{F}$. The requirement of a step σ is $\text{req}(\sigma) = \text{req}(p) \cup \{\text{bind}\}$ if σ binds any earlier step, and $\text{req}(p)$ otherwise.

Definition 2.4 (well-capability). A plan $\mathcal{P}$ over sources $\{S_i\}$ is *well-capable* iff for every step σ with source S , $\text{req}(\sigma) \subseteq \text{cap}(S)$.

2.3 Honest declaration, and its asymmetry

The critical property of Theorem 2.1 is what happens when the declaration is wrong, and it is not symmetric.

Proposition 2.5 (under-declaration is safe, over-declaration is not). *Let S have true capability set C^* and declared set cap .*

- (a) *If $\text{cap} \subsetneq C^*$, then some plans that the source could in fact serve are refused. The refusal is visible, names the missing capability, and is repairable by amending the declaration. No wrong answer is produced.*
- (b) *If $\text{cap} \supsetneq C^*$, then a plan passes the static check, the request is issued, the source returns something — a well-formed result set — and that result set is wrong in a way no field of the output records. The failure is invisible.*

Proof. (a) The check of Theorem 2.4 is a subset test computed before execution; a missing element yields a refusal naming $\text{req}(\sigma) \setminus \text{cap}(S)$. (b) The check passes by construction, so no refusal is emitted; the adapter’s lowering produces a request the source accepts under a reading the adapter did not intend, and the extraction returns whatever that reading yields. Nothing in the pipeline compares the returned rows against the intended semantics, because there is no independent statement of the intended semantics to compare against. $\square$

Remark 2.6 (the honesty assumption, and the largest gap). Theorem 2.5(b) is the single largest gap in this design and we do not close it. The capability declaration is data written by a human. A declaration claiming **neg** for a source that silently ignores negation produces precisely the confident wrong answer that motivates the whole apparatus. The design’s answer is architectural rather than technical: the declaration is small (eleven bits), local, reviewable, and versioned alongside the adapter, so that it is the kind of artefact a reviewer can actually check. That is a mitigation, not a proof, and a reader should weigh it as such.

A concrete instance is worth stating because it inverts the usual intuition.

Proposition 2.7 (a supported operation that must be withheld). *Let a source expose a SPARQL endpoint with a server-side materialisation **cap**: aggregate queries over result sets exceeding the **cap** return a well-formed aggregate computed over a truncated set, with an HTTP 200 and no warning. Then an honest declaration must exclude **agg** from **cap**, even though SPARQL supports aggregation and the request succeeds.*

Proof. The declaration in Theorem 2.1 asserts that the source can *perform* the operation, not that it will *accept* a request naming it. A truncated aggregate is not the aggregate; it is a different number returned under the same name. Including **agg** places the deployment in case (b) of Theorem 2.5, whose failure is invisible by construction. Excluding it places the deployment in case (a), whose failure is visible and repairable — the plan is refused, and the plan author computes the aggregate client-side over rows the source *can* return faithfully. $\square$

This is the sharpest single point we can make to a chemist about why the capability vocabulary is not bureaucratic overhead. Under the conventional pipeline the truncated aggregate is a number that appears in a table in a paper. Under this one, the plan does not run.

2.4 Budget and allocation

Steps cost requests, and a plan declares a budget. When the budget binds, something must give, and *how* it gives is a semantic decision, not an engineering one.

Definition 2.8 (yield). The yield of step i is a function $\gamma_i : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}_{\geq 0}$ from effort, measured in requests, to expected returned cardinality. We assume γ_i is non-decreasing, continuously differentiable, strictly concave, with $\gamma_i(0) = 0$.

Theorem 2.9 (water-filling allocation). *Under Theorem 2.8, the allocation problem $\max \sum_i \gamma_i(e_i)$ subject to $\sum_i c_i e_i \leq B$, $e_i \geq 0$, has a unique optimum e^* characterised by a single scalar shadow price $p^* \geq 0$ with $\gamma'_i(e_i^*) = p^* c_i$ for every i with $e_i^* > 0$, and $\gamma'_i(0) \leq p^* c_i$ otherwise.*

Corollary 2.10 (no dropping rule). *No step is excluded from the allocation by heuristic, priority order, or blacklist. A step receives zero effort only when its marginal yield at zero is below the shadow price, which is a statement about the step and the budget jointly, and is reported as such.*

Theorem 2.10 matters for the same reason the rest of this does. A federation engine that drops a subquery under load returns a result set that is missing rows for a reason the caller cannot see. Here, a step that received no effort is reported with a **blocked-by-budget** blocker, and the caller knows the difference between *there were none* and *we did not look*.

Remark 2.11 (where concavity fails). A **lookup** against a keyed REST service is all-or-nothing: one request returns the record or nothing, so γ is a step function and not concave. Theorem 2.9 does not apply to it. Such steps are charged first at their fixed cost and the remainder is optimised over the concave steps. We note this rather than pretending the assumption is universal.

3 Six verdicts, and why two are not enough

3.1 The rules

Each executed step receives exactly one verdict, assigned by the first matching rule of an ordered list. Ordering is what makes the assignment total and deterministic.

Definition 3.1 (verdict rules). For a step σ with source S , bound inputs Γ , allocation e , and outcome O :

- R1 If $\text{req}(\sigma) \not\subseteq \text{cap}(S)$: **unsupported**, blocker **blocked-by-model**. The source cannot perform the operation.
- R2 If $e = 0$ under a binding budget: **exhausted**, blocker **blocked-by-budget**. Effort was not allocated.
- R3 If some bound input Γ_j is empty: **starved**, blocker **blocked-by-corpus**. The step was unreachable; it was not attempted.
- R4 If the source declined or errored: **refused**, blocker **blocked-by-engine**.
- R5 If the request completed and returned no rows: **empty**, *no blocker*.
- R6 Otherwise: **answered**, no blocker.

Proposition 3.2 (totality and determinism). *The rules assign exactly one verdict to every executed step.*

Proof. R6 is unconditional, so the assignment is total; the rules are consulted in order and the first match returns, so it is unique. $\square$

3.2 The absent blocker is the point

R5 assigns no blocker, and this is a design decision that a reader should push on rather than accept.

Principle 3.3 (the blocker map is partial on purpose). **empty** carries no blocker, and the corresponding JSON field is *absent* rather than null. An empty result that completed is not blocked by anything. The corpus was consulted, the constraint was evaluated, and the extension is empty. That is a fact about the chemistry.

Every other verdict is a fact about the apparatus. Keeping the distinction in the shape of the output — a missing key, not a null — means a consumer that reads **blocker** and finds nothing has been told something, and a consumer that reads **null** has been told nothing and may believe it was told something.

This is the resolution of Theorem 1.1. Situations (a)–(d) of Section 1 receive **empty** (no blocker), **unsupported** (blocked-by-model), **starved** (blocked-by-corpus), **refused** (blocked-by-engine) respectively.

Corollary 3.4 (the distinction is not recoverable downstream). *No SPARQL query, however written, recovers the **empty** / **starved** / **unsupported** / **refused** distinction from a result set, because the distinction is a property of the execution and the result-set interface has no channel for it. Recovering it requires changing the interface, which is what a plan-level verdict does.*

3.3 Starvation is the characteristic federated failure

Proposition 3.5 (starvation is reachable only under federation). *Over a single closed corpus, R3 cannot fire on a plan whose steps are all satisfiable: an empty bound input is itself an empty extension, and the composite is empty by R5. Under federation with a translation between namespaces, a step may be unreachable because a mapping — not the corpus — yielded nothing for the identifiers at hand.*

This is why **starved** has its own verdict rather than being folded into **empty**. *Nothing in the sequence store lacks a cysteine and no identifier we hold could be translated into the sequence store's namespace* are different problems, and only the second is fixed by adding a mapping.

We measure translation with two independent quantities.

Definition 3.6 (retention and amplification). For a translation map μ and a set S of identifiers in its domain namespace, the retention is $r_\mu(S) = |\{s \in S : \mu(s) \neq \emptyset\}|/|S|$ and the amplification is $a_\mu(S) = |\bigcup_{s \in S} \mu(s)|/|\{s \in S : \mu(s) \neq \emptyset\}|$, undefined when the denominator is zero. By convention $r_\mu(\emptyset) = 1$.

Proposition 3.7. *Retention and amplification are independent, and neither is determined by output cardinality.*

Proof. A map sending four of eight identifiers to one image each has $r = 0.5$, $a = 1$, output 4; a map sending two of eight to two images each has $r = 0.25$, $a = 2$, output 4. Equal outputs, different (r, a) . $\square$

Question Q1 (Section 6.1) exercises this: its translation step from enzyme identifiers to sequence accessions records $r = 0.75$, $a = 1.0$. One in four candidate enzymes has no sequence record. A conventional pipeline reports four candidates in and three out, and the missing one is indistinguishable from a filtered one.

4 Refusal before contact

Corollary 4.1 (refuse before contact). *If a plan is not well-capable, execution halts at the check stage and `requests_issued` = 0.*

Two aspects of this are stronger than they may first appear, and both are load-bearing.

4.1 The whole plan, not the failing step

Principle 4.2 (ill-capability is a property of the plan). When one step fails the capability check, the satisfiable steps do not run either.

The argument is not conservatism. Suppose a question imposes two constraints, one evaluable and one not, and suppose the evaluable half runs and emits its rows. The caller now holds a set of datasets that satisfy constraint A and an *unenforced* constraint B, and no field in the payload distinguishes them from datasets satisfying both. The partial result is more dangerous than no result, because it has the shape of an answer. Section 7.1 exhibits this concretely.

4.2 The refusal names the real obstacle

Principle 4.3 (name the obstacle). A refusal states which step, which source, which capabilities were required, which were declared, and which were missing.

Listing 2: The emitted refusal for G8. `requests_issued` is 0.

```
{ "refusal": { "plan": "dcat_g8",
  "outcome": "refused statically",
  "reason": "ill-capability plan; no request was issued",
  "failures": [ { "step": "uvvis", "source": "INST",
    "required": [ "neg", "pattern" ],
    "declared": [ "batch", "bind", "link", "lookup" ],
    "missing": [ "neg", "pattern" ] } ],
  "steps": [] }
```

A chemist reading this learns something actionable: the instrument registry is a keyed record service, and the question requires asking it to classify its own contents, which it cannot do. The fix is to add a source that can, or to reformulate. Compare what the conventional pipeline offers for the same situation: an empty result, and an afternoon.

5 When every step succeeds and the answer is still wrong

This section contains the argument we consider most important, and it is the one that is hardest to make to an audience that has internalised the reasoner.

5.1 The problem

A plan can be answered at every step, return exactly the rows its predicates specify, and answer a different question than the one asked. The mismatch is not between a step and a source — which is what the verdict algebra measures — but between the *question's logical form* and the *corpus's*. Nothing in Theorem 3.1 can detect it, because every rule is about the fate of a request.

Example 5.1 (substrate scope). *What is the substrate scope of BVMO-Y?* The corpus records two substrates on which BVMO-Y was run. A plan retrieves both, every step answers, and the emitted rows are correct. But *scope* is a claim about substrates the enzyme *would* accept, including ones nobody has tried. The returned set is the recorded extension; the question asked for an intension. These differ, and no traversal of this corpus closes the gap.

The gap in Theorem 5.1 is inductive: from observed instances to a general claim. That step is chemistry — structure-activity reasoning, homology, mechanism — and it is not inference over the recorded facts.

5.2 Why a reasoner makes this worse

Here is the sharpest divergence between the two methods, and it runs against the intuition that more inference is more help.

Proposition 5.2 (asserting the closure launders an assumption). *Let an ontology assert an axiom sufficient to derive substrate scope from recorded substrates — for instance, that the substrate class of an enzyme is closed under a structural similarity relation. Then the reasoner derives triples asserting acceptance of untested substrates, and those derived triples are indistinguishable, in the materialised graph and in any query result over it, from triples recording observed acceptance.*

Proof. Materialisation writes entailed triples into the same graph as asserted ones, in the same vocabulary. A SPARQL query matching the substrate predicate matches both. Distinguishing them requires provenance annotation on every derived triple and a query discipline that consults it, neither of which the closure operation provides by default. $\square$

The situation is therefore: the honest answer is *the corpus records two substrates and cannot tell you the scope*; the reasoner’s answer is *a list*, and the list is a mixture of observation and assumption that looks homogeneous. The chemist who reads that list has been given a prediction wearing the clothes of a record. This is not a hypothetical failure mode; it is the ordinary consequence of materialising a closure over sparse experimental data, and it is exactly why Reiter [1978, 1980] treated closed-world assumption and default inference as things to be marked rather than absorbed.

5.3 The construct

We add a fourth answer shape. It sits beside the verdict, not inside it.

Construction 5.3 (admissibility statement). A plan may emit

emit X with provenance as extension of "TEXT" because GAP

where $GAP \in \{\text{induction, vocabulary, conditions}\}$. The emitted document then carries an **admissibility** record stating what was asked for, that what is returned is the recorded extension, that **answers_question: false**, and a reason determined by the gap.

Two features of Theorem 5.3 are deliberate.

First, the verdicts stay honest. The steps did answer; recording **refused** here would falsely suggest retrieval failed, and a reader could not distinguish a corpus that lacks the rows from one that has them and cannot generalise over them. So **answers_question: false** appears *alongside verdict: answer*.

Second, the gap is named by the plan author from a closed set, not generated. A free-text explanation is the same defect one level up: a canned sentence that sounds like reasoning and is not checked. A closed set of three is small enough to review and forces a claim about *which kind* of gap this is.

- **induction** — the question asks about cases the corpus does not contain (Theorem 5.1).
- **vocabulary** — the question uses a term the corpus does not record as such, though it may record something adjacent.
- **conditions** — the question asks for an outcome under conditions the corpus records for a *different* configuration.

Section 7.2 shows that **vocabulary** covers two structurally distinct cases — a missing type on a subject the store holds, versus an entirely absent relation — which is a limitation of the current three-way partition that we name rather than paper over.

6 Five questions from a working group

This section works the five biocatalysis questions end to end. They were supplied by a practising group, not constructed by us, and we did not alter them. Where a question cannot be answered we say so and say why; where our first attempt was wrong we show the wrong version.

The federation comprises six sources, each declaring a capability set under Theorem 2.1: **RXN** (reactions and catalysis, a graph source), **TAX** (organism attribution), **SEQ** (protein sequences, a keyed record service), **ACT** (activities and runs, a provenance source), **INST** (an instrument registry, keyed records), and **DS** (dataset descriptions). One translation map, $\mu : \text{enz} \rightarrow \text{seq}$, crosses the enzyme and sequence namespaces.

6.1 Q1: bacterial transaminase without cysteine

Which biocatalyst, originating from a bacterium (not a eukaryote), catalyses the transamination of benzylethylamine and does not have a cysteine in its protein sequence?

This is one sentence and three sources. Catalysis is in **RXN**; organism attribution is in **TAX**; residue composition is in **SEQ**, whose identifiers are accessions and not enzyme identifiers. Listing 1 gives the plan. Its execution:

step	verdict	n	r_μ / a_μ	note
enzymes	answered	5	—	catalysts of the transamination
bacterial	answered	4	—	one eukaryotic entry excluded
candidates	answered	4	—	intersection
keyed	answered	3	0.75 / 1.0	namespace translation
no_cys	answered	2	—	residue filter

Table 1: Q1. Three requests issued, budget twelve, no early halt.

Two enzymes are returned, and that is the answer. What we want to draw attention to is the fourth row.

Four candidates entered the translation and three left it. The step reports $r_\mu = 0.75$: one in four candidate enzymes has no sequence record in the accession namespace. It also reports $a_\mu = 1.0$: no enzyme mapped to more than one accession, so no candidate was silently duplicated.

Under the conventional pipeline this step is a join, and a join that loses a row loses it silently. The chemist sees four candidates in one intermediate result and two in the final, and has no way to apportion the attrition between *filtered by the cysteine constraint* and *never reached the sequence store*. Those have different remedies. The first is a finding. The second is a missing cross-reference, and it is fixed by curation, not by chemistry.

Remark 6.1. This is the only step in all thirteen plans that carries map metrics, because it is the only step that crosses a namespace. We state this rather than leaving the impression that retention and amplification are reported everywhere. Where there is no translation there is no attrition to measure, and the fields are absent — not zero.

6.2 Q2: buffer and pH of a specific methyl transfer

Which buffer composition and pH was used in the biocatalytic methyl transfer using methyl-transferase mt-X of Bacillus subtilis?

Five steps, five requests, all **answered**: two *B. subtilis* enzymes, of which one is MT-X; one methyl-transfer reaction; two runs; two condition records. The plan returns the buffer and the pH.

The interesting part is the organism clause, and it is interesting because of what happens when we falsify it. We ran the plan a second time with the organism constraint changed to an organism that does not host MT-X. The `mt_x` step went **empty** and every downstream step went **starved**. The plan returned nothing, and it returned nothing *with a blocker naming the corpus*.

Principle 6.2 (a constraint is only a constraint if the answer depends on it). For each constraint in a question, falsifying it must change the emitted payload. A constraint that can be falsified while the payload is unchanged is not being enforced; it is a comment on the plan.

We applied Theorem 6.2 to every constraint in every plan. It caught a real defect in our own work, in Section 7.3.

6.3 Q3: substrate scope and product range

What is the substrate scope and product range of the Baeyer–Villiger monooxygenase BVMO-Y?

Three requests, three steps, all **answered**: two reactions, two substrates, two products. Every step succeeded and the rows are correct.

And the plan does not answer the question. This is Theorem 5.1. The corpus records substrates BVMO-Y *was run on*; the question asks for the substrates it *would accept*. The plan therefore emits

```
emit substrates with provenance
    as extension of "substrate scope" because induction
```

and the emitted document carries, beside the verdicts:

```
"admissibility": {
  "asked_for": "substrate scope",
  "returned": "recorded extension",
  "answers_question": false,
  "reason": "... the inductive step from observed instances to
    scope is chemistry, not inference, and asserting it in an
    ontology would launder an assumption into a derived fact." }
```

Note that **verdict** remains **answered** on every step while **answers_question** is **false**. Both are true. The retrieval succeeded; the question was not answered. Collapsing these into a single field would force us to lie about one of them.

6.4 Q4: expected products under stated conditions

What are the expected products of a biocatalytic kinetic resolution reaction with enzyme PFE at pH 9 in HEPES buffer?

We flagged this in Section 1 as the one for a chemist to read first. Here is the execution, verbatim from the evidence table:

step	verdict	<i>n</i>	blocker
<code>pfe_reactions</code>	answered	1	—
<code>pfe_runs</code>	answered	1	—
<code>conditions</code>	answered	1	—
<code>at_ph9</code>	empty	0	<i>none</i>
<code>in_hepes</code>	starved	0	blocked-by-corpus

Table 2: Q4. Three requests issued. The last two rows are the result.

Read the last two rows carefully, because they are two different things and the difference is the paper.

at_ph9 is **empty with no blocker**. The corpus holds a PFE kinetic resolution and holds its conditions, and those conditions are not pH 9. The apparatus worked perfectly; the answer is that the experiment at this pH is not in the record. That is a fact about the chemistry, and Theorem 3.3 is why the field is absent rather than null.

in_hepes is **starved** with blocker **blocked-by-corpus**. It was not attempted. Its input was empty, so it was unreachable. This is a fact about the plan’s structure, not about HEPES: we learn nothing about whether the corpus records HEPES buffers, because we never asked.

Now consider what the received pipeline returns for this question. An empty solution sequence. One bit. The chemist cannot tell whether PFE was never run under kinetic resolution, whether it was run but not at pH 9, whether it was run at pH 9 but not in HEPES, or whether the buffer vocabulary simply is not recorded — and those four have four different next actions, one of which is *run the experiment*.

Proposition 6.3 (a reasoner makes Q4 worse). *Suppose the ontology carries axioms sufficient to predict products of a kinetic resolution from substrate and enzyme class. Then a reasoner materialises product triples for the PFE reaction, and the query returns them. The returned products are predictions under an assumed mechanism, in the same vocabulary and the same graph as recorded products, and the result set does not distinguish them.*

This is Theorem 5.2 instantiated on the question the chemist most wants answered, and it is where the two methods diverge most sharply. The reasoner’s answer is a list of products. Ours is: the corpus has the reaction, has the conditions, and the conditions are not the ones you named — reported as **empty** without a blocker, which is to say *this is your chemistry, not our plumbing*. The reasoner’s list is more useful right up until it is wrong, and it carries nothing that lets the reader find out which.

6.5 Q5: instrument and wavelength for a dated observation

With which device was the UV spectrum monitored during biocatalytic transformation BT3 on March 23rd by Yuliia Dikova, and which wavelength was monitored?

Two requests, four steps, all **answered**: two monitored activities, of which one is BT3, one on the date, one by the operator. The plan returns the device and the wavelength.

Three constraints here are non-trivial — transformation identity, date, operator — and we falsified each in turn. Changing the date sends **on_date** to **empty** and **by_ydikova** to **starved**; changing the operator sends **by_ydikova** to **empty**. Each constraint is on the path from the question to the payload.

6.6 An implementation defect the framework caught in itself

While building the sources for Q5 we found that a predicate expressing *monitored on date d* matched nothing, for every date. The cause was in our own adapter: the graph pattern applied its literal restriction to the *subject* position, so a literal appearing as an object matched nothing at all. The step reported **empty** — and **empty** carries no blocker, which is to say the framework told us *the corpus has no such thing*, confidently, and was wrong.

We report this because it is the framework’s own central failure mode reproduced inside its implementation, and because it bounds what Theorem 3.3 can promise. A blockerless **empty** means *the request completed and returned nothing*. It does not mean *the request asked what you meant*. That second guarantee is not available at this layer, from us or from anyone, and a reader who takes **empty** as evidence of absence is taking on the correctness of the adapter’s lowering as an assumption. The remedy we actually used was Theorem 6.2: the defect surfaced because a constraint we expected to matter did not change the payload when falsified.

7 Eight dataset-discovery patterns

The second group of questions is generic Chem-DCAT-AP dataset discovery: *give me all datasets D about...* These are the patterns a catalogue is supposed to serve, and they are a fairer test of the standard than the bespoke questions, because they are what it was designed for.

	pattern	req.	rows	outcome
G1	about a substance with compound C	2	2	answered
G2	generated by activity of type T evaluating C	2	2	answered + vocabulary
G3	... with Bruker spectrometer set to X	0	—	refused statically
G4	about reaction R with product P	4	2	answered
G5	... with a starting material of compound C	4	2	answered
G6	... with a solvent of compound C	3	2	answered + vocabulary
G7	... with a catalyst of compound C	5	2	answered
G8	measured with a UV-vis spectrometer...	0	—	refused statically

Table 3: The eight generic patterns. “req.” is `requests_issued`.

7.1 G3 and G8: refused at zero requests

Both refusals have the same shape and G8 is the sharper one, so we take it. The question asks for datasets measured with a UV-vis spectrometer. The instrument registry `INST` declares `{lookup, link, bind, batch}`: it returns a record given a device identifier, and it accepts a batch of them. The step requires `{pattern, neg}`, because *which of your devices are of kind K* is a pattern match over the registry’s contents, not a lookup by key. The check fails, and Listing 2 is emitted with `requests_issued = 0`.

Here is the part that matters. **The data to answer this question exists in the federation.** The corpus contains device `DEV:UV1900i` and activity `ACT:BT3`, and Q5 (Section 6.5) successfully retrieves that very device. We are refusing a question whose answer is present.

That is not a defect, it is the design working. Q5 asks *which device did this activity use*, which is a link from a known activity. G8 asks *which of your devices are UV-vis spectrometers*, which is a classification over the registry’s extension. The registry can do the first and cannot do the second; the fact that one particular device happens to be reachable by the first route does not make the second route available. A pipeline that answered G8 by enumerating the devices it happened to have seen would return a subset of unknown coverage, formatted identically to a complete answer.

Principle 7.1 (the refusal is about capability, not about data). A refusal states that no source can evaluate the constraint. It makes no claim that the answer is absent. These are independent, and G8 exhibits a case where the first holds and the second does not.

Theorem 4.2 is also visible here: G8’s other steps are perfectly satisfiable, and none of them ran. Had they run, the caller would hold datasets satisfying the compound constraint and *not* the instrument constraint, in a payload whose shape does not distinguish them from datasets satisfying both.

7.2 G6: a vocabulary the corpus does not have

G6 asks for datasets about a reaction whose *solvent* has compound C. Every step answers; two datasets are returned; and the plan carries a **vocabulary** admissibility statement, because the corpus does not record a solvent relation at all. It records reaction media. The returned rows are about media, and media are not solvents.

The tempting repair is an ontology axiom: declare `buffer` a subproperty of `solvent`, and the reasoner supplies the missing relation. This is worth taking seriously because it is what the received method would actually do, and it is wrong on the chemistry. The solvent of a 50 mM

Tris buffer is water; the buffer is a solute system dissolved in it. An axiom equating the two produces triples asserting that the solvent of the reaction is Tris, and those triples enter the graph indistinguishable from observations (Theorem 5.2). The catalogue then answers G6 — fluently, and falsely.

Table 3 marks G2 with the same vocabulary gap, and the two are not actually the same case: G2’s subject is present in the store and lacks a type, whereas G6’s relation is absent entirely. Our gap taxonomy has three values and does not separate these. We record that as a limitation of the present partition rather than claiming the three-way split is complete.

7.3 G7, and a defect in our own plan

G7 asks for datasets about a reaction whose *catalyst* has compound C. Five steps, five requests, all **answered**, two datasets. The answer is correct.

Our first version of this plan was also all-**answered** and also returned those two datasets, and it was wrong. It had a step that identified the catalyst and checked its compound, and then a final step that retrieved datasets bound to the *reactions found before the check* rather than to the checked set. Every verdict was **answered**. The rows were right.

We found it by Theorem 6.2. Falsifying the catalyst identity — substituting an enzyme that is not the catalyst of that reaction — sent the **named** step to **empty**, exactly as it should, and the final step *still emitted two datasets*. The plan was answering *datasets about this reaction*, dressed as an answer to *datasets about this reaction with this catalyst*.

The repair is structural, and the plan carries the reasoning in a comment because the next person to edit it needs it:

```
# The reactions that survive the check, reached FROM the confirmed
# catalyst. Binding 'cat_rxns' here instead would make the check
# decorative: the counterfactual run beside this one falsifies the
# catalyst, 'named' goes empty, and a plan that then binds
# 'cat_rxns' still emits two datasets -- a correct answer to
# "datasets about this reaction" dressed as an answer to "datasets
# about this reaction with THIS catalyst". Every constraint must sit
# on the path from the question to the payload, or it is not a
# constraint, it is a comment.
let confirmed = from RXN
  ask catalysed_reactions(?c)
  with ?c in named
  within 20
```

Principle 7.2 (a checked constraint that nothing depends on is invisible). Verdicts classify what happened to each request. They cannot detect that a step’s output is unused downstream, because that is a property of the plan’s dataflow, not of any request. Only falsification exposes it.

We report this at length because it is a limit on our own claims. The verdict algebra is not a correctness check on plans. It tells you what happened to each step; it does not tell you that the steps compose into the question you asked. The counterfactual discipline of Theorem 6.2 is the part that does that work, and it is methodology, not machinery — it must be performed, and nothing in the executor enforces it.

7.4 One further contrast worth stating

G6 and G7 are the same question shape — *datasets about a reaction one of whose participants has compound C* — differing only in the participant role. G7 answers cleanly and G6 does not, and the entire difference is that someone recorded a catalyst relation and nobody recorded a solvent relation.

That is worth saying plainly to a catalogue designer. The difference between a question this infrastructure answers and one it cannot is not depth of reasoning, expressiveness of the ontology,

or power of the engine. It is whether the relation was written down. Every layer above the record inherits that, and a layer that appears to escape it — by deriving the missing relation from an axiom — has not escaped it, it has stopped reporting it.

8 Two design decisions that look like omissions

Two things a reader coming from SPARQL will expect to find, and will not. Both are deliberate, and both follow from Theorem 1.1 rather than from expedience.

8.1 There is no set difference in the plan language

The plan language has `union`, `intersect`, `join ...on`, `filter ...where`, and `map ...via`. It has no `minus`. SPARQL has `MINUS` and `FILTER NOT EXISTS`, and their absence here is the single largest expressive difference between the two languages.

The reason is that a plan-level difference operator would let a plan manufacture a negative answer out of two positive ones, and thereby destroy exactly the distinction the verdicts exist to preserve.

Proposition 8.1 (a plan-level difference erases starvation). *Let A and B be steps, and suppose a plan may write $A \setminus B$ as a plan-level operation over their payloads. If B is **starved** — unreachable, never issued — then its payload is empty and $A \setminus B = A$. The combined step is therefore indistinguishable, in both payload and cardinality, from the case where B was reached and genuinely matched nothing.*

Proof. Immediate: $A \setminus \emptyset = A$ regardless of why B 's payload is empty, and the plan-level operator sees only payloads. The verdict on B survives in the step record, but the *payload* the caller reads carries no trace of it, and any downstream step binding the difference inherits the erasure rather than the verdict. $\square$

This is Theorem 1.1 reappearing one level up, inside our own language, which is why we regard it as disqualifying rather than merely inconvenient. The alternative we adopt is that negation is a *source capability*, `neg`, declared by a source that can actually evaluate an absence over its own extension. A step requiring `neg` either lands on a source that has it — in which case the absence is evaluated where the data live, by something that knows the extent of what it is quantifying over — or the plan is refused. G8 is this case: the instrument registry does not declare `neg`, and the plan is refused at zero requests (Section 7.1) rather than answered by differencing two things the registry *can* return.

The cost is real and we state it. Questions that a SPARQL author would write with `FILTER NOT EXISTS` over a single materialised graph must here be either pushed into a source that declares `neg`, or refused. On a federation where every source is a full SPARQL endpoint over a complete graph, this buys nothing and costs expressiveness. On the federation the chemist actually has — REST sequence services, keyed instrument registries, a local provenance store — it is the difference between a negative answer that means something and one that means *we did not look*.

8.2 Comparability is a property of a pair, not of a dataset

Q2 and Q5 look like retrieval questions and are usually treated as such: *which buffer*, *which device*. But the reason a chemist asks them is almost never to learn a buffer identity in isolation. It is to decide whether two runs may be compared — whether this transformation and that one were carried out under conditions close enough that a difference in outcome is about the chemistry.

That target is not a property of either dataset. It is a relation on the pair, and it is not one an ontology can store, for a reason that is arithmetic rather than architectural: with n runs there are $\binom{n}{2}$ pairs, and the relation depends on a tolerance the asker supplies at question time. A catalogue

that pre-computed comparability would have to fix the tolerance before knowing the question, and would be storing a quantity that grows quadratically and is invalidated by every new run.

We do not solve this here, and we flag it as the most promising place for the plan layer to go next, because the plan layer is where it naturally sits. A plan already carries steps whose inputs are bound by earlier steps; a comparability step is one whose input is bound by *two*, with a tolerance as a parameter, and whose verdict would distinguish *these runs are not comparable* from *the record does not say whether they are comparable* — which is again Theorem 1.1, in the place where a chemist would most be misled by its collapse. What blocks us is not the plan machinery but the representation of the tolerance itself, which is the open problem recorded under Section 12’s measurement paragraph.

9 Full evidence

Every number quoted in this paper comes from the table below, which is the executor’s own per-step output over the thirteen plans. We give it in full rather than in summary so that a reader can check the claims against it. “req.” is `requests_issued`; “wc” is well-capability.

plan	step	n	verdict
Q1	req. 3, halted false, wc true		
	enzymes	5	answered
	bacterial	4	answered
	candidates	4	answered
	keyed	3	answered
	no_cys	2	answered
$(r_\mu = 0.75, a_\mu = 1.0)$			
Q2	req. 5, halted false, wc true		
	bs_enzymes	2	answered
	mt_x	1	answered
	mt_reactions	1	answered
	runs	2	answered
	settings	2	answered
Q3	req. 3, halted false, wc true		
	reactions	2	answered
	substrates	2	answered
	products	2	answered
Q4	req. 3, halted false, wc true		
	pfe_reactions	1	answered
	pfe_runs	1	answered
	conditions	1	answered
	at_ph9	0	empty (no blocker)
	in_hepes	0	starved (blocked-by-corpus)
Q5	req. 2, halted false, wc true		
	measured	2	answered
	bt3	1	answered
	on_date	1	answered
	by_ydikova	1	answered
G1	req. 2 rxns 1, datasets 2 — all answered		
G2	req. 2 rxns 1, datasets 2 — all answered, gap vocabulary		
G3	req. 0, halted true, wc false brukers unsupported		
G4	req. 4 products/has_p/rxns 1, datasets 2 — all answered		
G5	req. 4 inputs 2, has_c 1, rxns 1, datasets 2 — all answered		
G6	req. 3 rxns 1, datasets 2, media 2 — all answered, gap vocabulary		
G7	req. 5 cat_rxns/catalysts/named/confirmed 1, datasets 2 — all answered		
G8	req. 0, halted true, wc false uvvis unsupported		

Table 4: Per-step evidence for all thirteen plans.

Three facts in Table 4 deserve to be read off explicitly, because they are the paper’s empirical content and a summary would hide them.

First, **Q1.keyed** is the *only* row in the entire suite carrying retention and amplification. Where no namespace is crossed there is no attrition to measure and the fields are absent. We say this rather than leaving a reader to infer that map metrics are pervasive, which they are not.

Second, **G3** and **G8** are the only rows with **requests_issued** = 0, and they are exactly the rows with **well_capability** false. That correspondence is Theorem 4.1 holding operationally: the static check is the thing that halts, and it halts before contact.

Third — and this is the one worth dwelling on — **eleven of the thirteen plans have every step answered**. The verdict algebra spends almost all of its time saying *nothing went wrong*. That is not a weak result, it is the correct shape for a working federation, and it is worth stating because a reader could otherwise take this paper as claiming that federated retrieval is mostly broken. It is not. The apparatus earns its keep on the two rows in **Q4** and the two refusals, and those five rows out of the roughly forty in the table are precisely the ones a one-bit interface would have rendered as a single undifferentiated absence.

10 Validation, and what it does not establish

Sixteen checks (V1–V16) run over the compiler and the executor. They cover: that the verdict assignment is total and deterministic; that the blocker map is partial exactly at `empty`; that a plan failing the capability check issues no request; that allocation is feasible and exhausts the budget when demand exceeds it; that no step is dropped by rule (Theorem 2.10); that retention and amplification agree with Theorem 3.6 on constructed maps; that the blame chain terminates; that emitted documents round-trip through JSON with the blocker field *absent* rather than null; and that the parser rejects the constructs it is specified to reject. All sixteen hold.

The round-trip check deserves a note, because it is the least interesting-looking and guards the most. Theorem 3.3 says the blocker map is partial at `empty`. A serialiser that emitted `"blocker": null` would satisfy every other check while quietly reintroducing a distinguished absent value — the exact move that Section 12 criticises SQL’s NULL for. The check asserts the key is missing, not that its value is null.

Remark 10.1 (the disclaimer that matters more than the result). V1–V16 test the implementation against the theory. They do not test the theory against the world. That a verdict is assigned deterministically says nothing about whether it is the right verdict; that no request is issued for an ill-capable plan says nothing about whether the capability declaration was true. The check that would matter most — does a source’s declaration match its behaviour — is precisely the one we cannot run, for the reason in Section 11.

There is a second gap the suite cannot reach, and Section 7.3 is the evidence for it: no check detects a plan whose steps all succeed and which answers a different question. That defect is invisible to everything in V1–V16, and it was found by hand. We would rather report sixteen checks with both caveats attached than a larger number that implies coverage we do not have.

11 On not probing a third party’s endpoint

The evaluation runs against a local fixture, constructed to mirror the structure of the group’s records — the same sources, the same namespace split, the same sparse recording — rather than against a live public endpoint. Two reasons, and the second is the binding one.

The practical reason is reproducibility. A paper whose numbers depend on a third party’s deployment state reports something unrepeatable: the endpoint changes and Table 4 becomes unverifiable.

The reason we regard as binding is ethical. Characterising a source’s *true* capability set — which is what Theorem 2.6 says we cannot verify — would require issuing requests designed to elicit internal behaviour: probing for materialisation caps, for silent truncation, for the point at which an aggregate stops being the aggregate. That is adversarial traffic against a public research service maintained on a shared budget, and it is not ours to send. We hold to this even though it leaves the largest gap in the design unclosed. Given the choice between naming a gap and closing it by that route, we name it.

There is a related observation we can report but not explain. On 10 August 2026, two requests to a public endpoint, byte-identical except for one line, returned 2 and 397 rows respectively — a ratio of 198.5. We do not know why. Every hypothesis we can form is a hypothesis about the deployment’s internals, and discriminating among them requires exactly the probing above. So it stands here as an undiagnosed observation rather than a characterised behaviour. We think reporting it undiagnosed is more useful than either omitting it or speculating, and we note that it is the kind of thing Theorem 2.7 is a response to: a client that cannot rule out truncation should decline to declare the capability, not guess at the threshold.

The implementation reflects the constraint in code rather than in intention. The adapter module carries it in its docstring: every adapter resolves against a local fixture or a local engine; no adapter performs network I/O, and none may be added that does.

12 Related work

Source capability description. Describing what a source can do is old and well studied: mediator architectures [Wiederhold, 1992], source descriptions for query answering [Levy et al., 1996], and capability-aware optimisation across heterogeneous sources [Haas et al., 1997]. SPARQL federation inherits the problem [Prudhommeaux and Buil-Aranda, 2013], systems such as FedX [Schwarte et al., 2011] plan around source capability, and the space is surveyed by Saleem et al. [2016]. Nothing in Section 2 is novel as *description*, and we do not claim it is.

The difference is what happens to a capability mismatch. In all of the above it is a planner-internal fact: the planner routes around it, rewrites, or fails, and the caller receives a result set. Here it is a first-class output — named, with the required set, the declared set, and the difference, at `requests_issued` = 0. We are not claiming a better planner. We are claiming that the mismatch is information the caller needs, and that the conventional interface has nowhere to put it, which is Theorem 1.1.

Buil-Aranda et al. [2013] is close in spirit: it surveys the reliability of public SPARQL infrastructure and finds it uneven. Theorem 2.7 can be read as asking what a client should do about that finding, and answering: declare the capability away, and accept refusals in exchange for never receiving a truncated aggregate that has the shape of a complete one.

Nulls and certain answers. The relational literature has long distinguished the reasons a value or a row may be absent [Codd, 1979, Imielinski and Lipski, 1984], and Libkin [2016] shows how far SQL’s three-valued treatment sits from certain-answer semantics. Our Theorem 3.1 is a coarser instrument aimed at a different level: not *is this value unknown* but *did this step get to look*. The connection is that both are refusals to let one distinguished value — NULL there, the empty solution sequence here — carry more meanings than it can separate. The debt is direct, and Theorem 3.3 is the place we try hardest to honour it: rather than introducing a distinguished “no blocker” value, which would recreate the problem, we make the field absent.

Reasoning. We use description-logic reasoning [Baader et al., 2003, Grau et al., 2008, Motik et al., 2009] as the foil throughout, and we want to be precise about the criticism, because it is narrower than the framing might suggest. Nothing here says a reasoner computes the wrong entailments; under its axioms it computes exactly the right ones. The criticism is about *materialisation into a shared vocabulary*: entailed and asserted triples become indistinguishable to any query over the result (Theorem 5.2), and on sparse experimental data the entailed fraction can be large while its axioms go unexamined by the person reading the answer. The closed-world and default-reasoning literature [Reiter, 1978, 1980] treats exactly this as something to mark rather than absorb, and our admissibility statement is a coarse, plan-level version of that marking — coarser than a default theory, and correspondingly cheaper to write and to read.

The standard and its domain. Chem-DCAT-AP extends DCAT-AP [European Commission, ISA Programme, 2015, Albertoni et al., 2020] with catalysis vocabulary, in the FAIR tradition [Wilkinson et al., 2016] and the biocatalysis standardisation effort around it [Dörr et al., 2020, Pleiss, 2021, Bornscheuer et al., 2012]; the sources it federates are the usual ones [The UniProt Consortium, 2023, Wittig et al., 2018, Chang et al., 2021, Hastings et al., 2016], with provenance in PROV-O [Lebo et al., 2013], and a query layer specified by Harris and Seaborne [2013] over engines such as Tanon [2024]. We repeat that we keep all of this. The present paper changes the answering layer, not the vocabulary.

Measurement. One connection we flag without developing. The metrology guide [Joint Committee for Guides in Metrology, 2008] requires a measured value to carry its uncertainty. RDF does not: a triple asserting pH 9.0 and a triple asserting pH 9.0 ± 0.5 are structurally the same triple with different literals, and no query over them can be made to respect the difference. Q4 sits

directly on this — *at pH 9* is a sharp cut through a quantity that was measured with a tolerance — and so does the comparability question of Section 8.2, which is *entirely* a question about tolerances. We do not solve it here. We note that the framework this work builds on handles it by making every value carry a strictly positive floor, so that a sharp cut is *untypeable* rather than merely discouraged [Sachikonye, 2026], and that reconciling a typed floor with RDF literals is open.

13 Limitations

Beyond the four limits stated in Section 1, three more have emerged from doing the work, and they are sharper than the ones we knew to state in advance.

- (i) **The verdicts do not check plans.** Section 7.3 is our own defect: every step answered, the right rows, the wrong question. The algebra classifies what happened to each request, and a constraint whose output nothing consumes is a dataflow property that no verdict can see. The remedy we used — counterfactual falsification of every constraint (Theorem 6.2) — is methodology. Nothing in the executor enforces it, and a plan author who skips it gets no warning. A static check that every non-terminal step’s output is transitively consumed by the emitted step would catch this particular shape, and we have not built it.
- (ii) **The gap taxonomy is under-resolved.** Three values, and Section 7.2 shows vocabulary covering two structurally different situations — a missing type on a subject that is present, and a relation that is absent from the schema entirely. A finer partition would be more informative and harder to assign consistently. We have not found the right cut, and we would rather ship three honest values than five that the plan author assigns by coin flip.
- (iii) **A blockerless empty inherits the adapter’s correctness.** Section 6.6 is the demonstration: our own lowering asked the wrong question, and the framework reported *nothing there*, confidently. **empty** means the request completed and returned nothing. It does not mean the request expressed the constraint the plan author intended. That second guarantee is not available at this layer, and a reader who treats **empty** as evidence of absence is taking the adapter’s lowering on trust.

14 Conclusion

Thirteen questions from working biocatalysis practice, put to a plan language rather than to a query language. Eight return rows. Two are refused before any request is issued, one of them for data that demonstrably exists in the federation — and that refusal is correct, because no source can evaluate the constraint and a partial answer would have had the shape of a complete one. Two return rows under a named admissibility gap. One returns **empty** without a blocker and a downstream starved with one, which is the distinction the whole apparatus exists to preserve.

We keep the ontology. The change is at the answering layer, and the argument for it is a single observation: a result set carries one bit of failure information, and laboratory catalysis data — partial by nature, federated by necessity, unevenly recorded by circumstance — routinely produces empty results for reasons that one bit cannot separate. Distinguishing them is not a matter of writing better queries, because the distinction is a property of the execution and the interface has no field for it.

What we can promise is narrower than the tally suggests, and we would rather end on that than on the tally. The apparatus reports faithfully what happened to each request. It does not certify that the requests expressed the question — our own G7 plan returned the right rows for the wrong reason, and our own adapter reported an empty corpus that was not empty. It does not verify a source’s capability declaration, and it deliberately declines the only route that could. What it does is smaller and, we think, worth having: when this infrastructure fails to answer a

chemist’s question, the document it returns says whether the failure was in the chemistry or in the plumbing. That is one bit more than the alternative, and it is the bit the chemist needs in order to know whether the next move is to fix a cross-reference or to run the experiment.

References

- Riccardo Albertoni, David Browning, Simon Cox, Alejandra Gonzalez Beltran, Andrea Perego, and Peter Winstanley. Data catalog vocabulary (DCAT) — version 2. W3C recommendation, World Wide Web Consortium, 2020.
- Franz Baader, Diego Calvanese, Deborah McGuinness, Daniele Nardi, and Peter Patel-Schneider. *The Description Logic Handbook*. Cambridge University Press, 2003.
- Uwe T. Bornscheuer, G. W. Huisman, R. J. Kazlauskas, S. Lutz, J. C. Moore, and K. Robins. Engineering the third wave of biocatalysis. *Nature*, 485:185–194, 2012.
- Carlos Buil-Aranda, Aidan Hogan, Jürgen Umbrich, and Pierre-Yves Vandenbussche. SPARQL web-querying infrastructure: ready for action? In *International Semantic Web Conference*, pages 277–293, 2013.
- Antje Chang et al. BRENDA, the ELIXIR core data resource in 2021. *Nucleic Acids Research*, 49 (D1):D498–D508, 2021.
- E. F. Codd. Extending the database relational model to capture more meaning. *ACM Transactions on Database Systems*, 4(4):397–434, 1979.
- Mark Dörr, Detlef Hasenpusch, and Uwe T. Bornscheuer. Standardisation and automation in biocatalysis: towards machine-readable experimental records. *Biotechnology Journal*, 15(8): 1900407, 2020.
- European Commission, ISA Programme. DCAT application profile for data portals in europe. Technical report, Publications Office of the European Union, 2015. Version 1.1.
- Bernardo Cuenca Grau, Ian Horrocks, Boris Motik, Bijan Parsia, Peter Patel-Schneider, and Ulrike Sattler. OWL 2: the next step for OWL. *Journal of Web Semantics*, 6(4):309–322, 2008.
- Laura M. Haas, Donald Kossmann, Edward L. Wimmers, and Jun Yang. Optimizing queries across diverse data sources. In *Proceedings of VLDB*, pages 276–285, 1997.
- Steve Harris and Andy Seaborne. SPARQL 1.1 query language. W3C recommendation, World Wide Web Consortium, 2013.
- Janna Hastings, Gareth Owen, Adriano Dekker, Marcus Ennis, Namrata Kale, Venkatesh Muthukrishnan, Steve Turner, Neil Swainston, Pedro Mendes, and Christoph Steinbeck. ChEBI in 2016: improved services and an expanding collection of metabolites. *Nucleic Acids Research*, 44(D1): D1214–D1219, 2016.
- Tomasz Imielinski and Witold Lipski. Incomplete information in relational databases. *Journal of the ACM*, 31(4):761–791, 1984.
- Joint Committee for Guides in Metrology. Evaluation of measurement data — guide to the expression of uncertainty in measurement. Technical Report JCGM 100:2008, BIPM, 2008.
- Timothy Lebo, Satya Sahoo, and Deborah McGuinness. PROV-O: The PROV ontology. W3C recommendation, World Wide Web Consortium, 2013.

- Alon Y. Levy, Anand Rajaraman, and Joann J. Ordille. Querying heterogeneous information sources using source descriptions. In *Proceedings of VLDB*, pages 251–262, 1996.
- Leonid Libkin. Three-valued logic and certain answers in SQL. *ACM Transactions on Database Systems*, 41(1):1–28, 2016.
- Boris Motik, Rob Shearer, and Ian Horrocks. Hypertableau reasoning for description logics. *Journal of Artificial Intelligence Research*, 36:165–228, 2009.
- Jürgen Pleiss. Standardized data, FAIRness, and the reproducibility of enzyme kinetics. *Chem-CatChem*, 13(18):3924–3929, 2021.
- Eric Prudhommeaux and Carlos Buil-Aranda. SPARQL 1.1 federated query. W3C recommendation, World Wide Web Consortium, 2013.
- Raymond Reiter. On closed world data bases. In *Logic and Data Bases*, pages 55–76. Plenum Press, 1978.
- Raymond Reiter. A logic for default reasoning. *Artificial Intelligence*, 13(1–2):81–132, 1980.
- Kundai Farai Sachikonye. Federated querying over heterogeneous chemical knowledge sources: Plans, capabilities, and verdicts. Technical report, HEGEL/CKG, 2026.
- Muhammad Saleem, Yasar Khan, Ali Hasnain, Ivan Ermilov, and Axel-Cyrille Ngonga Ngomo. A fine-grained evaluation of SPARQL endpoint federation systems. *Semantic Web*, 7(5):493–518, 2016.
- Andreas Schwarte, Peter Haase, Katja Hose, Ralf Schenkel, and Michael Schmidt. FedX: optimization techniques for federated query processing on linked data. In *International Semantic Web Conference*, pages 601–616, 2011.
- Thomas Pellissier Tanon. Oxigraph: a graph database implementing the SPARQL standard, 2024. Software.
- The UniProt Consortium. UniProt: the universal protein knowledgebase in 2023. *Nucleic Acids Research*, 51(D1):D523–D531, 2023.
- Gio Wiederhold. Mediators in the architecture of future information systems. *Computer*, 25(3):38–49, 1992.
- Mark D. Wilkinson et al. The FAIR guiding principles for scientific data management and stewardship. *Scientific Data*, 3:160018, 2016.
- Ulrike Wittig, Maja Rey, Andreas Weidemann, Renate Kania, and Wolfgang Müller. SABIO-RK: an updated resource for manually curated biochemical reaction kinetics. *Nucleic Acids Research*, 46(D1):D656–D660, 2018.